

CS3217 Notes

github.com/gerteck

Pre-Course Readings and Resources

Some suggested or referenced readings and watchings for the course.

- **Clean Code**, by **Robert C Martin**, **The Mythical Man Month**, **Rework** by **37signals**

Some notes for the course: CS3217 is designed to be a very intensive software engineering module, write lots of code. First 6 weeks - 4 problem sets to develop some Peggles by end of 6 weeks. Remaining period, work in teams of up to 4 students for the Final Project. **Final Project:** For the Final Project, allowed to build app of choice, start thinking early, see past projects for scale. Develop a good app that solves problem defined. Good to spend time to build a real product that is useful.

Stay Hungry. Stay Foolish.

- Steve Jobs' 2005 Stanford Commencement Address. Three personal stories:
- **Connect the Dots:** Life events often only make sense retrospectively, trust in the journey and intuition. (Seemingly impractical knowledge may turn out useful.)
- **Love and What You Do:** Passion for work drives excellence and satisfaction, give motivation to start over and seek success.
- **Death as a Motivator:** Awareness of mortality helps prioritise and live authentically.
- **Stay Hungry, Stay Foolish:** Maintain curiosity and willingness to take risks and learn.

Paul Orfalea's Keynote Address.

- Founded Kinko's (now part of FedEx). Had to overcome learning disabilities and reading writing difficulties.
- **Pragmatism:** Education focus on applying knowledge to real life over memorization or taking extensive notes (rip... tbf...).
- **Leadership:** Warned against leaders who cannot tolerate ambiguity or delegate authority, noting they risk burnout or nervous breakdown.
- **Work-Life:** Balance work, love and play to avoid burnout and maintain effective leadership, anxiety and ambition are intertwined, sleep important for retention, decision making, cultivate personal happiness and relationships.
- Many fail to see the success around them as they are too busy or anxious.
- Overall, important to leverage unique strengths, focus on human relationships, balance life priorities for sustainable success.

Randy Pausch's The Last Lecture

- Persistence and creativity are key to overcoming obstacles and achieving dreams.
- Fun and enthusiasm fuel sustained effort and creativity.
- Collaboration between disciplines (art and tech) is vital for innovation.
- Leadership and humility, and support systems of family, mentors and colleagues are crucial for success.

Problem Set 0

Ungraded problem set to learn Swift and SWE. UI development is not the focus of this problem set. Build a Sudoku Solver Program

Lecture 1: Introduction

- CS3217: Software Engineering. CS3216: Product Development.
- Software Engineering: Understanding the Problem, Articulating Needs, Design, Reasoning about Correctness, Processes (Working with people, maintaining code, coping with change).
- Language matters not. (But it does in terms of properties and suitability.)
- **Problem Sets:** Learn SWE, learn Apple/Android, ccomplete app at the end, Peggles clone and achieve individual competency.

Plan: First 7 weeks (including recess week) - 4 problem sets, last 7 weeks - Final Project.

- **Education:** Preparing students for a job that do not yet exist in a future we cannot predict. Most valuable skill is how to learn efficiency. How to figure out what you don't know and build on what you do know to adapt to new situations and new problems.
- **Environment → Experience:** shaped by surroundings.
- **Engineering:** SWE: systematic engineering approach to software development. Engineering: use of scientific principles to design and build machines, structures, and other items.
- **A well engineered product:** Maintainable at its core / crux of engineering (especially for companies with software as a core product offering). Other aspects include functional, adaptable, within budget, timely, efficient.
- **That which cannot be maintained, will eventually die.**
- 'If you are not prepared to be wrong (or at least, not the best), you will never come up with anything original.' - Ken Robinson

Lecture 1b: Introduction to Swift

- **Comparison:**
 - **Java/C#:** Established, traditional OOP, "Language matters not".
 - **Kotlin/Swift:** Modern, designed from scratch, Avant-garde POP (Protocol Oriented Programming), "Language matters much".
 - **Why Study PLanguages?:** Medium of our industry (Civil Engineering analogy). Toolchain: Xcode, Playground.

Swift

- **Typing:** Swift is Strongly and statically typed.
- **Collection Types:** Arrays, Sets, Dictionaries, **Tuples** (kind of like an immutable array that can hold different types).

Case Study: Swift Strings

- **Evolution of String dataType:** Swift 1: Collection → 2: Non-Collection → 4: Collection → 5: UTF-8. **Why?**
- **Context and definitions:**
 - *ASCII* is major standard that maps 0-127 to english characters, is 1 byte (7 bits + 1 pad), can only represent English.
 - *Unicode:* Not an encoding of bits, but a giant list / map that assigns unique code points (numbers) to characters across all languages. (E.g. 'A' is U+0041).
 - *UTF-8:* Modern standard, variable-width encoding of Unicode code points. Backwards compatible with ASCII (0-127 same as ASCII), uses min of 8 bits (1 byte), up to max 32 bits (4 bytes) per code point. (Note UTF-16 uses 2 or 4 bytes per code point).
- **Evolution of Swift Strings:** Comparison between correctness and usability.
- In Swift 1, Strings as collections of characters, used UTF-16 directly, but acted as fixed-width arrays, directly jumping with subscripting chars[5] dangerous (due to variable-width encodings).
- In Swift 2, String as non-collections, correct for variable-width encodings, but hard to use (no subscripting, no iteration).
- In Swift 4, Strings back as Collections of Characters (revisited). Balance correctness, usability with Character type (grapheme clusters). But still using UTF-16 fixed box storage.
- **Swift 5:** Strings as UTF-8 Collections. Improved performance and interoperability while maintaining correctness and usability.
- **Graphemes:** User-perceived characters. E.g., 'é' can be 'U+00E9' or 'e' + 'U+0301'. Swift handles canonical equivalence (displays identically, compares equal).
- **Emojis:** May not fit in single UTF-16 unit. Swift counts grapheme clusters correctly (length 1), unlike Java or C#. In many languages, a complex Emoji is just a list of numbers. In Swift, it is a single Character.
- **Iterating:** (The 'No Integer Indexing' Rule): As Swift treats variable-width blobs as single characters, does not know where char 5 lives without checking preceding characters.

Legibility of Swift

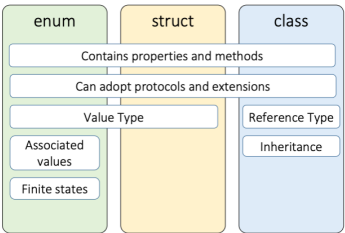
- **Syntax:** Semicolons optional, brackets optional, braces required. Legibility focus.
- **Optionals:** 'Type?' != 'Type'. NOT the same as null in Java. *Forced Unwrapping (!):* Dangerous, runtime error if nil. Use *Optional Chaining* instead. a?.b?.c. If any link is nil, result is nil. Result is always optional.
- **Optional Binding (if let, else):** Unwraps into a new constant. (E.g. if let pOwner = cat.owner?.name).
- **Guard (guard let):** Enforces "Golden Path" (indented scope for happy path not required). Avoid "Pyramid of Doom". Exits early if nil.
- **Functions:** Function parameters are labelled, which are required unless omit with .. Parameters are constants by default, use inout to make mutable. Functions are also first-class citizens.

Swift Programming Paradigms

- **Structs vs Classes:** Structures same as classes, except classes support inheritance, type cast, deinit, ref counting. So why even use structs?
- Structs are value types (copied on assignment), classes are reference types (reference counted). Structs on Stack, faster, Classes allocated on Heap, slower. Structs also more safe as it is copied, no shared mutable state, thread safety etc.
- **When to use Structs vs Classes:**
 - **Structs (Value Types):** No inheritance, no deinit, no reference counting. Automatic member-wise initializer. Copied on assignment. *Use for things that are values.*
 - **Classes (Reference Types):** Inheritance, reference counting, identity. *Use for things that have identities.*

– Use Structs by default.

- **Enumerations:** Group of related values of any type, first-class types, has computed properties, methods, initializers.



Protocol Oriented Programming (POP)

- **Protocols:** A protocol defines a blueprint of methods, properties, and other requirements that suit a particular task or piece of functionality (Interfaces).
- **Extensions:** Add functionality (computed props, methods) to existing class, structure, enumeration or protocol type (even those you don't own).

Lecture A: Names, Comments, Testing (Clean Code)

What is Clean Code?

- **Definition:** Code that is easy to read, understand, and modify.
- Clean code is *elegant, efficient*, and **does one thing well**. (Bjarne Stroustrup). Clean code reads like *well-written prose*. (Grady Booch) Clean code looks like it was written by **someone who cares**. (Michael Feathers)
- **The Boy Scout Rule:** Always leave the campground cleaner than you found it. Encourages continuous refactoring and prevents code rot.

Meaningful Names

- **Names should reveal intention:** Code should be understandable without comments. Good naming explains *what* and *why*.
- **Avoid disinformation:** Do not encode container types into names (e.g. accountList). Focus on the *concept*, not the implementation.
- **Make meaningful distinctions:** Avoid vague variations (e.g. data, data2, dataInfo). Differences in names must reflect differences in meaning.
- **Pick one word per concept:** Avoid mixing terms like *controller, manager, director* arbitrarily. Consistency reduces cognitive load.
- **Names should be searchable:** Single-letter names are hard to search. (Acceptable only in small local scopes).
- **Rule of thumb of Name Lengths**
 - Name length should correspond to scope size.
 - Larger scope → longer, more descriptive names.
- **Avoid mental mapping**
 - Prefer meaningful names over short cryptic ones.
 - Readers should not need to translate names mentally.
- **Nouns and Verbs: Classes:** Nouns (things). **Functions:** Verbs (actions).

Meaningful Comments

- **Why comment?:** Code should ideally be self-explanatory. Comments last resort, not a crutch.
- **Comments do not fix bad code:** Do not comment to explain messy code. Refactor and clean the code instead.
- **Good comments**
 - Good comments explain **intent**.
 - Clarify unavoidable complexity (with care).
 - Warn of consequences (e.g. performance, safety).
 - Amplify the importance of non-obvious logic.
- **Risks of clarifying comments:** Can become outdated or incorrect. Errors are hard to detect.
- **Bad comments:** Raise more questions than they answer, repeat what code already says, or clutter the codebase. Are misleading or factually incorrect.
- **Commented-out code:** Should be removed. Version control already preserves history.
- **Prefer code over comments:** Replace comments with well-named variables or functions.

Testing

- **Why test?:** Prevent catastrophic failures. Increase confidence that the system behaves as intended.
- **Case Study: Ariane 5:** Self-destructed 37 seconds after launch, due to software error of reused code of integer overflow from Ariane 4. Cost: Over \$1 billion.
- **Root causes:** Code reuse without revalidation, assumption that it worked before. and missing tests for new operational context.

Verification vs Testing

- **Verification:** Are we building the system correctly?
- **Testing:** Are we building the correct system?

Testing Taxonomy

- **Unit Testing:** Individual modules work correctly.
- **Integration Testing:** Modules work together correctly.
- **Validation Testing:** System meets specifications.
- **System Testing:** Works with external systems.

Test Driven Development (TDD)

- **TDD is a discipline:** Follows strict, somewhat arbitrary rules. Justified by long-term correctness and design quality.
- **Three Laws of TDD**
 1. Do not write production code until a failing test exists.
 2. Do not write more test code than needed to fail.
 3. Do not write more production code than needed to pass.

- **TDD Loop:** Short feedback cycle (~5 seconds). At any moment, code worked a minute ago.
- **Guidelines:** Don't tackle the core goal first. Write tests that define all the other random edge cases and etc. and then work towards the core goal.
- **Benefits:** Minimal debugging required. Code is designed to be testable → decoupled. Tests act as executable documentation.

Tests as Documentation

- Tests document APIs and usage examples.
- Written in the same language as the system.
- One-way dependency: tests depend on code, not vice versa.
- Best form of low-level documentation.

Test Quality Matters

- Dirty tests are worse than no tests.
- Poorly written tests reduce trust in the suite.
- Gaps in tests invalidate confidence (parachute analogy).
- Tests must evolve alongside production code.

Tests counter Fear of Refactoring

- Main fear: breaking existing functionality.
- Good test suites remove this fear.
- Enables continuous refactoring and clean design.

Why write Tests First?

It is fun when Tests pass! (Endorphines). You cannot write code that is hard to test, making it decoupled and you have a button that you can trust.

- Programmers write a document with strange symbols nobody can read, every symbol must be correct or terrible things happen.
- Note that TDD: is a skill, if new to it and immediately try to apply, will give up immediately.

Guidelines for Good Tests

- **Dual Standard:** Tests do not need to be as efficient as production code. (Test env is different from production env).
- **One concept per test**
- **One assert per test** (guideline, not dogma)
- **F.I.R.S.T Principles:**
 - **Fast:** Tests should run quickly.
 - **Independent:** Tests should not depend on each other.
 - **Repeatable:** Same result in any environment.
 - **Self-validating:** Pass or fail, no manual checking.
 - **Timely:** Written just before production code.

Problem Set 1: Graph ADT

ADT (Abstract Data Type) Specifications

Specifications: The purpose of a specification is to document a program's logic, the range of inputs that it can safely operate on, and its expected behaviour.

- Interpreting an ADT involves understanding its specifications - what it does and does not. So for ADTs, the specification describes what this ADT represents, its purpose, its representation invariant, and the behaviour of each of its methods.
- At the start of the ADT, the specification gives a summary of the ADT, mainly the abstraction function and the representation invariant. The abstraction function is a mapping from the state of an ADT instance to some mathematical object that it represents. The representation invariant is some condition that all valid instances of the ADT must satisfy, which is essentially the format of the ADT.

Lecture 2: Design Patterns and Functions

Functions

- **Goal:** Functions should be easy to read, reason about, test, and reuse. Clean functions are the foundation of clean code.
- **Functions should be small:** Optimally, a few lines, definitely no longer than a full screen. Smaller functions improve readability and testability.
- **BAD: Complicated & messy function:** function mixes multiple responsibilities (e.g. determine if page is test, setup, teardown, render HTML all in one). Too long, too many nested levels, multiple abstraction levels, repeated logic.
- **Refactoring Fixes:** Method extraction, introduce meaningful helper functions, improve readability, delegate details to helpers.
- High-level function should read like prose.
- **Curly's law:** Functions Should Do One Thing, do it well, do it only.
- **Single Level Abstraction Principle (SLAP):** Every step in a function should be at the same abstraction level. Error handling is considered a separate abstraction. Even improved versions can violate SLAP if abstraction levels are mixed.
- Ofc, don't be dogmatic about it.

Switch Statements

- **Problems:** Too large, does more than one thing.(E.g. determines the type of entity, decides which action to take based on type, performs the action).
- Violates **Single Responsibility Principle (SRP)** (Each function only one reason to change, but we change it if new type added, logic changes.)
- **Open-Closed Principle (OCP):** Open for extension, closed for modification. Switch statements violate OCP as new types require modifying existing code.
- **Then how? Solution:** One solution is to bury switch statements inside factories. Use polymorphism to remove conditional logic. (i.e. instead of using switch to determine type and action, delegate action to the object itself - each type implements XX interface).
- Localize the OCP violation to a single factory function. (Factory is the extension point)

Function Arguments (Parameters)

- **Less arguments, the better:** Arguments increase cognitive load. Hence, best functions have zero or few arguments. Niladic (0 arguments), Monadic (1 argument), Dyadic (2 arguments), Triadic (3 arguments, avoid), Polyadic (>3, require strong justification).
- **Confusing Argument Order:** Issue is e.g. confusing ordering of arguments etc. Labels (Swift) may help improve clarity.
- **Flag arguments are ugly:** `render(true)` hides intent. Better to have separate functions: `renderCompressed()` and `renderUncompressed()`. Prefer expressive function names.

Function Side Effects (Bad)

- Functions should not have side effects. Side effects are **lies**. When we call a function, we do not expect it to change the state of the system in unexpected ways.
- Function promises to do one thing, secretly does another. (E.g. `isPasswordValid(password)`, also initializes user session if valid, which is unexpected side effect).

Functions should not have Output Arguments

- **Output Arguments:** when function modifies parameter passed by reference and returns it.
- Problem is that it causes a lot of ambiguity.
- **Better:** Return new value instead of modifying input parameter.

Don't Repeat Yourself (DRY)

- Repetition leads to bugs and inconsistencies, extract shared logic into reusable functions.

What are Design Patterns

- **Design:** Strategy for organizing, implies some form of architecture.
- **Pattern:** Proven, repeatable solution, converged over time.
- Design Patterns are **reusable solutions** to common software design problems, evolve through widespread use and refinement.

Anti-Patterns

- **Late exit / unhappy path:** Logic buried inside deeply nested conditionals. Prefer early exit (e.g. with `guard` in Swift).
- **Blind faith (forced unwrapping):** Leads to runtime crashes, use `guard let` (in Swift) instead. Force unwrapping is only allowed in e.g. (IBOutlets and IBActions), or essential properties in `prepareForSegue`, where instant crash preferred if something wrong.
- **Bloat:** Too much logic in a single function. Refactor into meaningful sub-functions

Common Software Architectural Patterns

How are large enterprise systems designed? Choose some suitable architecture to provide desired functionality and quality attributes.

- **Macro Design:** Decide on high-level patterns, technologies (e.g. layered, microservices)
- **Micro Design:** Class/Method structure, lower-level patterns (e.g. Singleton, Factory).
- **Layered Pattern, (aka N-tier pattern):** Structure programs that can be decomposed into groups of subtasks at different levels of abstraction. Each layer provides services to the layer above it and acts as a client to the layer below it.
- (E.g. Presentation UI Layer, Application service layer, Business Logic Domain Layer, Data Access Persistence Layer.). Example usage include desktop apps, ecommerce web apps.
- **Client-server pattern:** Two parties; a server and multiple clients. Server component provides services to multiple client components. Clients request services from server, and server continues to listen to client requests. (E.g. online apps like email, document sharing, banking).
- **Master-slave pattern:** One master component delegates work to multiple slave components that perform the work and report back to the master. (E.g. parallel processing, distributed databases).
- **Pipe-filter pattern:** Series of processing elements (filters) connected by channels (pipes). Each filter processes data and passes it to the next filter. (E.g. Unix command line tools, compilers).
- **Broker pattern:** Structuring distributed systems with decoupled components that interact through remote service invocations. A broker component mediates communication between clients and servers.
- Broker component responsible for coordinating communication among components. Servers publish their capabilities to a broker, clients request services from the broker, which forwards requests to appropriate servers and returns results to clients. (E.g. message brokers e.g. Kafka, RabbitMQ, message-oriented middleware).
- **Peer-to-peer pattern:** Decentralized network architecture, each node (peer) can function as both client and server. Communicate directly with each other w/o relying on a central server. (E.g. file sharing networks, blockchain networks).
- **Event-bus pattern:** Deals with events, 4 major components: event source, event listener, channel and event bus. Components communicate by publishing and subscribing to events on particular channels via a central event bus. Publishers emit events to the bus, and subscribers register to receive specific event types. (Like an internal pub-sub system, e.g. game dev, unity, unreal, plugin architectures).
- **Model-view-controller pattern:** Separates application into three interconnected components: Model (data and business logic), View (user interface), Controller (handles user input, updates model and view). Promotes separation of concerns, modularity, and maintainability. (E.g. web applications, desktop applications).
- **Blackboard pattern:** Useful for problems which no deterministic solution strategy known. Components (knowledge sources) contribute to a common data structure (blackboard) to solve a problem collaboratively. Knowledge sources monitor the blackboard for changes and contribute new information or solutions when relevant. (E.g. speech recognition systems, complex problem-solving applications).
- **Interpreter pattern:** Defines a grammatical representation for a language and provides an interpreter to deal with this grammar. Used to evaluate sentences in the language. (E.g. SQL parsing, mathematical expression evaluation).

What Makes a Good Architecture?

A good architecture should have quality attributes: balanced distribution of responsibilities, testability, ease of use. This enables incremental development, reduces fear of refactoring, lowers maintenance cost. Less code is better code. All about managing complexity.

Known Design Patterns

Gang of Four: computer scientists Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides authored *Design Patterns: Elements of Reusable Object-Oriented Software* (1994). Cataloged 23 patterns of solving common design problems.

- **Criticism of patterns:** typical arguments against using patterns. Good to keep in mind.
- **Weak programming languages:** Some languages have built-in features that reduce the need for certain patterns. (E.g. first-class functions reduce need for Strategy pattern). The programming language chosen or used is the problem.
- **Inefficient solutions:** too dogmatic use of patterns can lead to inefficient solutions. (E.g. Singleton pattern can introduce global state, making testing and reasoning about code harder).
- **Unjustified pattern overuse:** Overuse of patterns can lead to unnecessary complexity and over-engineering. Not every problem requires a design pattern. (If all you have is a hammer, everything looks like a nail).

Classification of Patterns

Design patterns differ in complexity, level of detail, scale of application to system being designed. Most basic and low level patterns are idioms, while high level patterns are architectural. All patterns also can be categorized by intent or purpose.

- **Creational Patterns:** object creation mechanisms that increase flexibility and reuse of existing code. (E.g. Singleton, Factory, Builder)
- **Structural Patterns:** explain how to assemble objects and classes into larger structures while keeping these structures flexible and efficient. (E.g. Adapter, Composite, Decorator)
- **Behavioral Patterns:** take care of effective communication and assignment of responsibilities between objects. (E.g. Observer, Strategy, Command)

Creational patterns

These patterns provide various object creation mechanisms, which increase flexibility and reuse of existing code.

Factory Method

Abstract Factory

Builder

Prototype

Singleton

Structural patterns

These patterns explain how to assemble objects, classes into larger structures while keeping structures flexible and efficient.

Adapter

Bridge

Composite

Decorator

Facade

Flyweight

Proxy

Behavioral patterns

These patterns are concerned with algorithms and the assignment of responsibilities between objects.

Chain of Responsibility

Command

Iterator

Mediator

Template Method

Memento

Observer

State

Strategy

Visitor

Read at <https://refactoring.guru/design-patterns/catalog> here.

E.g. Factory Method

Provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created. Suggests to replace direct object construction calls (using the new operator) with calls to a special factory method.

- Objects are still created via the new operator, but only inside the factory method. Objects returned by the factory method are often referred to as products.
- Then, override factory method in subclasses to change class of products created.
- Code that uses the factory method works with the resulting products through their base interface. This way, the code remains independent of the concrete product classes.

Presentation Patterns (UI)

Presentation patterns, often used in Mobile application, define how responsibilities are structured to improve maintainability, testability, scalability. E.g. MVC, MVP, MVVM.

- Good architecture enforces separation of concerns, improves unit testability, reduces long-term maintenance cost, and prevents overly large UI components (e.g. Massive View Controllers).
- ELI5: Stop putting database code inside button click listeners etc.

Scope within the System Architecture

It is important to distinguish between *Presentation Patterns* and *System Architectures*.

- **System Architecture (Macro Level):** Patterns such as *Layered Architecture*, *Clean Architecture*, or *Hexagonal Architecture* dictate the structure of the entire application (e.g., separating the Data Layer, Domain Layer, and Presentation Layer).
- **Presentation Pattern (Micro Level):** Patterns like MVVM or MVC dictate the internal structure of the *Presentation Layer* only.

A robust application typically utilizes both: a high-level System Architecture to organize the backend/data flow, and a Presentation Pattern to organize the UI components. E.g. a *Clean Architecture* system, outermost "Interface Layer" is structured internally using **MVVM**.

MV(X) essentials: MVC, MVP, MVVM, VIPER

- **Overview:** All MV(X) patterns separate concerns into Model (domain data and business logic, which manipulates the data), View (UI rendering), and some intermediary component that mediates between Model and View. (Responsible for altering Model based on user input, updating View based on Model changes).
- Dividing entities allows for better understanding, reuse, and testability.
- **Model–View–Controller (MVC):** MVC separates an application into Model (domain data and business logic), View (UI rendering), and Controller (mediate between Model and View).
- **Usage:** Suitable for small applications, prototypes, or screens with limited logic where development speed is prioritised over testability. MVC traditionally used in iOS UIKit development, the View Controllers become 'Massive VCs' due to accumulating too much responsibility.
- **Model–View–Presenter (MVP):** MVP treats the view controller as a dumb passive View that forwards user events to a Presenter. The Presenter contains presentation logic and updates the View through an interface, with no direct View–Model dependency.
- Excellent testability, cost of increased boilerplate, manual binding between View, Presenter.
- **Model–View–ViewModel (MVVM):** MVVM introduces a ViewModel that represents UI state independently of the UI framework. The View binds to observable properties exposed by the ViewModel, which updates the Model and reflects changes back to the View.
- MVVM improves testability, reduces UI update code because of binding mechanisms (e.g. LiveData, StateFlow, Rx). However, reactive flows can be harder to debug if poorly structured.
- **VIPER:** VIPER further decomposes responsibilities into View, Interactor (business and data logic), Presenter (UI logic), Entity (data models), and Router (navigation).
- Strongest separation of concerns and testability but significant boilerplate and maintenance overhead, making it suitable primarily for large or complex applications.

Considerations and Choosing a Pattern

- **ELI5 Summary:**
 - **MVC:** (Triangle architecture) Controller, has routing logic, tells model what data to update, and view the flow (which view to use, navigation). View observes model directly and gets the data from model, exists coupling between view and model. (E.g. login screen click, controller tells model to fetch data, then selects which view to display).
 - **MVP:** (Bridge architecture) Presenter sits strictly between View and Model. View is passive and dumb, presenter tells view what to do. View never observes model changes. (E.g. login screen, click a link, view tells presenter, presenter tells model to fetch data, uses data to tell view how to update). Separated, so easier unit testing.
 - **MVVM:** (Reactive) ViewModel (Middleman) holds (UI state + UI logic), but never talks directly or holds reference to View (UI elements + rendering logic), sits there holding variables. View binds (observes) ViewModel variables, and updates reactively. Model also does not know ViewModel exists.
 - (E.g. login screen, click a link, view tells ViewModel, ViewModel tells model (business state + business logic) to fetch data, ViewModel (UI state) reacts to model changes, View (UI) auto-updates by observing ViewModel, and handles its own navigation of Views).
- **Platform Considerations:** iOS (UIKit): MVC, MVVM, VIPER; iOS (SwiftUI): MVVM;
- Android (XML / Compose): MVVM (recommended), MVP (legacy).
- **Overall:** No single best pattern. Simpler patterns favour dev speed, lower overhead, more structured ones improve testability and scalability at higher maintenance cost.
- Choose based on project constraints and team experience.

Lecture B: Specification & Abstraction

Software Development Processes

- **Waterfall Model:** A linear, sequential development process consisting of: Requirement Specification, Software Design & Architecture, Implementation & Integration, Testing & Validation, Deployment & Maintenance. Each phase must be completed before the next begins.
- **Spiral Model:** An iterative, risk-driven process with four main activities: Determine objectives & constraints, Identify and resolve risks, Development and testing, Plan the next iteration. Emphasizes early risk mitigation.
- **Agile Model:** An iterative and incremental process emphasizing rapid feedback: Design, Build, Test, Release, Configure.

Decomposition and Abstraction

- **Decomposition:** Breaking down a complex problem into smaller, manageable sub-problems. Each sub-problem is roughly at the same level of detail, can be solved independently, and can be combined to solve the original problem.
- **Abstraction:** Focusing on high-level specifications while hiding implementation details. Abstraction allows independence from specific implementations, enabling flexibility and easier maintenance.

Abstraction

- **Abstraction by Parameterization:** Common computation patterns are abstracted using parameters. E.g. $x \times x + y \times y$ can be abstracted as: $\lambda x, y : x * x + y * y$. This allows reuse and generalization.
- **Abstraction by Specification (Contracts):** Specifications as Contracts between client and implementer. Each procedure associated with specification defining how it can be used and what it guarantees.
- If procedure A calls procedure B, A relies only on B's specification (locality), not its implementation. B can be reimplemented without affecting A (modifiability).
- This simplifies design, facilitates change, and improves maintainability. Complicated specifications often indicate poor design.

Types of Abstraction

- **Procedural Abstraction:** Abstracting common computation patterns into procedures with well-defined specifications. Abstracts implementation from intent, should be minimalist (few constraints on usage), underdetermined (allow multiple implementations), deterministic (same input produces same output), and general (handles wide class of inputs).
- Total procedures defined for all legal inputs, partial procedures undefined for some inputs (must include REQUIRES clause). Procedure specifications include REQUIRES (conditions assumed true), RETURNS, MODIFIES, and EFFECTS.
- **Data Abstraction:** Abstract Data Types (ADTs) encapsulate data representation and operations. Data can only be accessed through its operations, improving maintainability and allowing representation changes for efficiency.
- An abstraction function maps concrete state to abstract state, while a representation invariant is a predicate that must always hold for the concrete representation. A checkRep function verifies the representation invariant. Rep invariant should be first thing articulated, not an afterthought.
- **Iteration Abstraction:** Instead of exposing internal structure, provide a way to iterate over elements. This allows changing the underlying representation without affecting clients.
- Iteration can be implemented in several ways: returning all elements at once (arrays), storing traversal state inside collection itself, using external iterator objects, or using closures. Must consider simplicity, efficiency, and control. Iterator-based approaches must handle tracking the current position, avoid duplicated work between hasNext and next, and ensure correctness when the underlying collection is modified, often by invalidating the iterator.
- **Type Hierarchy:** Organizing types into a hierarchy based on shared characteristics. Subtyping allows substituting objects of a subtype for those of a supertype, preserving behavior. This enables polymorphism and code reuse.
- **Polymorphic Abstraction:** Allows code to work over many types. Approaches include using type hierarchies (subtyping) or requiring only necessary operations (structural typing). Example: A sorting function that works for any type that supports comparison, even if the types are unrelated.

Lecture 3: Memory Management & Initialisation

Two related pillars of Swift’s safety model: **safe initialisation** and **automatic memory management**. Swift enforces many rules at compile time to prevent undefined behaviour such as using uninitialised memory or accessing deallocated objects. Understanding these rules is essential for writing correct, crash-free applications.

Initialisation

Initialisation in Swift is deliberately strict. Unlike some languages that allow partially initialised objects, Swift requires that all stored properties have well-defined values before they are used. This design eliminates an entire class of runtime errors.

Swift Variables

Swift enforces **definite initialisation**, where every variable must be initialised before it is read, and the compiler checks all possible execution paths. There are no implicit default values for variables, except for optionals which default to nil.

```
var message: String
if lectureStarted {
    message = "You should be taking notes."
}
print(message) // ERROR: variable 'message' used before initialized
```

Even of message may be initialised at runtime, compiler rejects code as there exists a path where it is not.

Initialisers

Initialisers are special methods whose sole responsibility is to put an instance into a valid, usable state. Swift treats initialisation as a safety-critical phase and restricts what code can do during it.

- Initializing applies to struct, enum, and class
- All stored properties must be initialised exactly once, and before use
- **What Does “Before Use” Mean:** self cannot be accessed, instance methods cannot be called, properties cannot be read before initialisation is complete.
- This prevents code from observing or operating on a partially initialised object.

Safe Initialisation

```
struct Color {
    var red: Int, green: Int, blue: Int

    init(red: Int, green: Int, blue: Int) {
        self.red = red
        self.green = green
        self.checkRep() // ERROR: self used before... initialized
        self.blue = blue
    }

    func checkRep() {
        assert(red > 0 && green > 0 && blue > 0)
    }
}
```

Calling methods before all stored properties are initialised is forbidden. Even though checkRep looks harmless, it implicitly accesses self.

Initialising Classes

Class initialisation is more complex due to inheritance. Swift enforces strict ordering to ensure subclasses cannot observe uninitialised superclass state (and vice versa).

Subclass Initialisation Order

```
class Car {
    var paintColor: Color
    init(color: Color) {paintColor = color}
}

class RaceCar: Car {
    var hasTurbo: Bool
    init(color: Color, turbo: Bool) {
        super.init(color: color) // ERROR: 'hasTurbo' not initialized
        hasTurbo = turbo
    }
}
```

Rule: A subclass must initialise all of its own stored properties before calling super. init.

Calling Methods During Initialisation

Calling instance methods during initialisation is dangerous bc. subclasses may override them.

```
class Car {
    init(color: Color) {
        paintColor = color
        fillGasTank()
    }
}
```

If fillGasTank is overridden, it may execute before subclass properties are initialised.

Overloading Initialisers

Swift distinguishes between two kinds of initialisers to control complexity and ensure safety.

Designated vs Convenience Initialisers

- **Designated Initialisers:** Primary inits, fully initialises all properties introduced by class
- **Convenience Initialisers:** Secondary, helper inits, provide defaults or alternative parameter lists (syntax: convenience init(params) {...})
- **Initializer Delegation Rules:** Strict delegation hierarchy:
 - Designated initialisers must call a designated initialiser of the immediate superclass
 - Convenience initialisers must call another initialiser in the same class
 - Convenience initialisers must eventually call a designated initialiser
- **Safety Checks:** All properties introduced by class must be init before delegating to super class. Designated inits must delegate to superclass before modifying any inherited property.
- A convenience init must delegate to another init before it assigns any property. Initializer cannot call any instance method, read property values, or refer to self before delegating.

Two-Phase Initialisation

Swift class init happens in two distinct phases to guarantee safety across inheritance chains.

- **Phase 1:** Each class initialises the properties it introduced. Memory becomes fully allocated
- **Phase 2:** Each class can further customise the instance (e.g. call methods, modify properties). Instance is fully valid.

Initializer Inheritance

Initialisers are not inherited by default. This prevents subclasses from accidentally exposing unsafe or incomplete initialisation paths.

- **Automatic Initializer Inheritance:** Automatic inheritance occurs only when Swift can prove it is safe.
- If subclass does not define any designated initialisers, it automatically inherits all of its superclass’s designated initialisers.
- If subclass overrides all superclass designated initialisers, it automatically inherits all of the superclass’s convenience initialisers.

Memory Management

Swift abstracts away manual memory management while still providing deterministic performance characteristics.

Stack vs Heap

- **Stack:** fast, automatic, limited lifetime
- **Heap:** flexible size, managed via reference counting
- Objects and closures live on the heap.

Manual Retain Release (MRR), Automatic Reference Counting (ARC)

- **Manual Retain Release:** Before ARC, Objective-C required programmers to manually manage ownership: retain to claim ownership, release to relinquish ownership. This was powerful but extremely error-prone.
- **Auto Reference Count:** ARC automates reference counting at compile time. ARC inserts retain/release calls automatically based on variable scope and assignments. Objects deallocate immediately when reference count hits zero.
- This gives Swift deterministic deinitialisation without runtime pauses. ARC allows us to write code naturally, not worry and concentrate on app. - ARC is not garbage collection, as it is deterministic and does not require pauses.
- **Garbage Collection:** Commonly refers to Mark and Sweep algorithm, periodically pauses execution to reclaim unreachable memory. (Used in Java, Python, Ruby). Mark live objects from root, sweep heap to free unmarked objects, no need to worry about cycles.
- GC (vs. ARC) requires 2-3 times more memory (android phones need more RAM), suspends execution during collection, unpredictable performance, but ease on programmer.
- ARC (vs. GC) has less memory overhead, deallocation compiled into program, deterministic deinitialisation, but programmer must manage reference cycles and leaks.

Swift Reference Types

Classes and closures are reference types in Swift. Variables store references (pointers) to heap-allocated objects.

- **Strong References:** Default in Swift, increase reference count
- **Weak References:** Do not increase reference count, must be optional type as becomes nil if object is deallocated.
- **Unowned References:** Do not increase reference count, non-optional. Assume referenced object always exists during lifetime of reference.

Reference	counting	var	let	optional	non-optional	Variable (var) vs Constant (let)
Strong	✓	✓	✓	✓	✓	let name: String
Weak	✗	✓	✗	✓*	✗	weak var apartment: Apartment?
Unowned	✗	✓	✓	✗	✓	unowned let customer: Customer

Reference Counting and Cycles

- **Reference Counting:** Each object has a reference count tracking how many strong references point to it. When reference count reaches zero, object is deallocated.
- **Strong Reference Cycles:** ARC cannot break cycles automatically. If two objects strongly reference each other, their reference counts never reach zero. This results in memory leaks.
- **Weak and Unowned References:** Swift provides two tools to break reference cycles. Weak References do not increase reference count, must be optional and auto set to nil if object deallocated. Unowned References also do not increase reference count, but are non-optional and assume referenced object always exists.

Choosing the Right Reference

- **strong** references → use for owners to objects they own.
- Both sides (properties) can be nil, optional → use **weak**. Use among objects with independent lifetimes.
- One side optional → use **unowned**. Use from owned objects with same lifetime.
- Neither optional → use **implicitly unwrapped optional**
- **Implicitly unwrapped optional:** use when property cannot be initialised immediately but is guaranteed to exist after initialisation. (E.g. var capital: City!). Behaves like non-optional after initialisation but allows deferred setup.

Closures and Reference Cycles

Closures are reference types and capture variables strongly by default.

- **Closures:** are reference types that capture variables strongly from their surrounding context.
- Common source of memory leaks, especially when stored as properties
- **Capture Lists:** used to control how values are captured (strong, weak, unowned)

```
lazy var closure = { [unowned self] in
    self.doSomething()
}
```

Reference Cycles in Structs

Although structs are value types, they can still participate in reference cycles if they store reference types.

- Cycles occur indirectly via heap-allocated objects
- Swift currently provides no direct mechanism to break them

This highlights that ARC issues are about references, not value semantics alone.

Lecture C: Value Type Semantics

Swift design philosophies: **value type semantics**. Use of value type encouraged to reduce bugs caused by unintended sharing, improve reasoning about code, and enable safer concurrency. Contrast value semantics with reference semantics, examine common pitfalls, and study how Swift achieves efficiency through copy-on-write. Swift’s model combines mutability where needed, with value semantics (immutability) by default and efficiency through copy-on-write. Understanding how to mix value and reference types correctly is key to writing robust Swift programs.

Reference Semantics vs Value Semantics

At a high level, understanding the distinction is crucial when designing APIs and data models.

- **Reference semantics:** multiple variables can refer to the same instance
- **Value semantics:** each variable has its own logically independent copy

Reference Semantics

Reference types (classes) allow shared mutable state. While powerful, this can lead to subtle bugs when changes in one place affect another unexpectedly.

Example: Temperature Class

```
class Temperature {
    var celsius: Double = 0
    var fahrenheit: Double {
        get { return celsius * 9 / 5 + 32 }
        set { celsius = (newValue - 32) * 5 / 9 }
    }
}
```

```
let temp = Temperature()
temp.celsius = 24
home.thermostat.temperature = temp
```

```
temp.celsius = 180
home.oven.temperature = temp
```

Classes are reference types, meaning assignments copy the reference, not the value. Both the thermostat and oven now point to the same `Temperature` instance. Mutating `temp` affects both — often unintentionally.

Copying as a Fix (and Its Problems)

One way to avoid unintended sharing is copying, but this approach scales poorly.

- **Manual Copying:** Programmer must remember to copy objects whenever sharing is undesirable. This relies on discipline and is error-prone.

```
home.thermostat.temperature = temp.copy()
home.oven.temperature = temp.copy()
```

- **Defensive Copying:** Classes defensively copy inputs in setters. Every setter must defensively copy. This pattern is tedious, error-prone, and does not compose well.

```
var temperature: Temperature {
    get { return _temp }
    set { _temp = newValue.copy() }
}
```

- **Framework-Level Copying:** Some frameworks (e.g. Objective C, Cocoa Touch) enforce copying via protocols or property attributes. However missed copies can still cause bugs.

Immutability

Immutability reduces bugs by preventing mutation altogether.

- **Example:** Make `Temperature` immutable by declaring properties as `let` and removing setters.

```
class Temperature {
    let celsius: Double
    var fahrenheit: Double { celsius * 9 / 5 + 32 }
}
```

- Now, once created, `Temperature` instances cannot be modified. This eliminates unintended sharing issues.

- While safer, immutable reference types can lead to awkward APIs that require creating new objects for small changes.
- **Immutability vs Performance:** Immutable data structures can have worse asymptotic performance:
 - Mutable algorithms: $O(n)$
 - Immutable rebuilding: $O(n^2)$

Value Semantics

Swift aims to balance safety and performance. Value semantics provide:

- Independent copies by default, Local reasoning about mutations and hence fewer unintended side effects
- **Logically distinct variables:** E.g. `var a = 17; var b = a; b += 25` means `a` remains 17, `b` becomes 42. Mutating `b` does not affect `a`.
- This behavior extends to user-defined types (structs, enums), strings, arrays, sets, and dictionaries.

Value Types in Swift

All fundamental Swift types are value types:

- Scalars: `Int`, `Double`, `Bool`
- Collections: `Array`, `Set`, `Dictionary`, `String`
- Custom types: `struct`, `enum`, `tuples`

Equality, Equatable Value Types

- Value types are compared by **value**, not identity. (E.g. `==` compares contents, not memory addresses, so two distinct strings with the same characters are considered equal).
- **Equatable:** Value types should conform to `Equatable`. This allows using `==` to compare instances based on their values.

```
struct Point {
    var x: Float
    var y: Float
}
```

```
extension Point: Equatable {
    static func ==(lhs: Point, rhs: Point) -> Bool {
        lhs.x == rhs.x && lhs.y == rhs.y
    }
}
```

- **Example:** Value-Type `Temperature`. Rewriting `Temperature` as a struct allows each assignment to create an independent copy, eliminating unintended sharing.

```
struct Temperature {
    var celsius: Double = 0
    var fahrenheit: Double {
        get { celsius * 9 / 5 + 32 }
        set { celsius = (newValue - 32) * 5 / 9 }
    }
}
```

- **Equatable with Reference type properties:** If a value type contains reference type properties, equality must be defined carefully to ensure logical equivalence. Identity comparison (`===`) is insufficient. Use deep equality to compare the contents of referenced objects. (E.g. `lhs.image.isEqual(rhs.image)`).

Let vs Var for Value Types

- `let` value type: cannot mutate any property
- `var` value type: mutation allowed

This enforces immutability at the variable level, not just the property level.

Value Semantics Benefits and Performance

- **Benefits:** Freedom from race conditions, safe mutation during iteration and easier reasoning in concurrent code.
- **Performance:** Copies are kept cheap via optimizations like copy-on-write. Small value types take constant time, large collections and data structures are copy on write. (Actual copy deferred until mutation).

Mixing Value and Reference Types

Mixing value and reference types can lead to unexpected behavior if not done carefully.

- **Reference in Value type:** If a value type contains a reference type property, copies of the value type will share the same reference. Mutating the referenced object affects all copies. Violates intuitive value semantics.
- **Value in Reference type:** If a reference type contains value type properties, each instance gets its own copy of the value-type properties. This generally preserves expected behavior.
- **Mutable References inside Value type:** If a value type contains mutable reference type properties, mutations to the referenced object affect all copies of the value type. This can lead to unexpected shared mutation.
- Use copy on write pattern to ensure mutations happen on private copies only.

When to Use Value types vs Reference types

Reference semantics enable sharing but risk unintended mutation. Value semantics provide safer defaults, better composability, and easier concurrency.

Use value types when:

- Equality matters more than identity
- Copies should be independent
- Data crosses thread boundaries

Use reference types when:

- Identity matters
- Shared mutable state is intentional

Tutorial 1: Data Structures, Error Handling, Language Design

Efficient Data Structures

- Choosing the right implementation matters: e.g., arrays vs linked lists for stacks and queues affect time complexity.
- Invariants and representation checks are crucial for correctness.
- Design decisions should balance performance, memory usage, and simplicity.

Abstract Data Type (ADT) Design

- Graphs and trees can be implemented with different representations depending on expected use cases (sparse vs dense, generality vs simplicity).
- Reusing existing ADTs can save effort, but sometimes a specialized implementation is clearer.
- Test-driven development ensures each operation meets specifications, guides cleaner design.

Error Handling Principles

- Different ways to handle errors: return special values, use optionals (nullable types), or throw exceptions.
- Choose a strategy based on whether errors are expected, recoverable, or exceptional.
- Explicit handling improves safety and maintainability; silent failures may lead to bugs.

Code Readability and Comments

- Comments should clarify **why** something is done, not **what** the code literally does.
- Redundant comments add noise and reduce readability.
- Clear code often reduces the need for explanatory comments.

Type Conversion and Language Design Choices

- Different languages handle type conversion differently, reflecting their design philosophy (safety vs convenience).
- Safe conversions (e.g., optional types, exceptions) encourage explicit handling of invalid input.
- When designing APIs or structs, favor approaches that make failure explicit to the caller, improving robustness.

Key Takeaways

- Focus on abstraction and correctness before optimization.
- Explicit error handling and careful ADT design improve code safety and maintainability.
- Immutability, clear invariants, and value vs reference semantics support safer concurrent and modular code.
- Write tests to guide implementation and verify correctness.

Tutorial 2: TDD, Functional and Imperative Paradigms

Test-Driven Development (TDD) and ADTs

- Define tests before implementing functionality; guides API design and ensures correctness.
- Consider edge cases, invalid input, and expected behaviors when designing tests.
- For game or domain ADTs (e.g., Tic-Tac-Toe), tests should cover state transitions, win/loss conditions, and invalid moves.
- TDD encourages modular and testable code.

Test Doubles and Dependency Injection

- **Test doubles** (mocks, stubs, fakes) simulate external dependencies to isolate unit under test.
- **Dependency injection** allows passing dependencies (e.g., databases, services) into components, making testing easier.
- Useful when external systems are slow, non-deterministic, or hard to set up.
- Less useful when dependencies are simple, stable, or tightly coupled to internal logic.

Single Level of Abstraction Principle (SLAP)

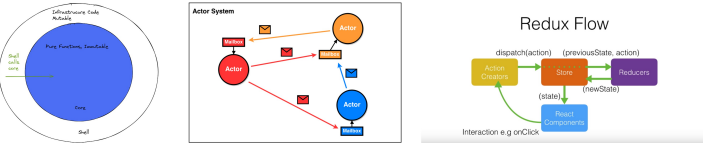
- Functions should operate at a single level of abstraction.
- Mixing details and high-level logic in a function makes code harder to read, test, maintain.
- Refactor long, complex functions into small, focused helper functions improves clarity, reuse.

Delegate Pattern and UI Design

- Delegates allow separation of concerns: UI components handle display, while domain logic is handled externally.
- Delegates are defined via protocols/interfaces specifying methods that UI component can call.
- Improves modularity, testability, and reusability of UI components.
- Example: A grid view component delegates user input handling to a separate class.

Functional Core, Imperative Shell Paradigm

- **Pattern:** Having a functional core where all pure functions (no side effects, immutable) live, and thin imperative shell where impure functions operate.
 - Core logic is **purely functional**: deterministic, no side effects, easy to reason about, test.
 - Shell handles **imperative concerns**: I/O, state mutation, user interaction, interact with outside world; usually thin and minimally tested.
- **Advantages of Functional Core, Imperative Shell:** Easier testing: functional core extensive tests. Predictable behavior and fewer bugs. Clear separation of pure logic and side effects.
- **Limitations:** Not always applicable for highly interactive or stateful systems where pure functional abstraction is difficult. Requires discipline to keep side effects confined to the shell.
- Related patterns: IO monad, Actor model, Flux/Redux architecture.
- **IO Monad (Functional Programming)**
 - Encapsulates side effects (e.g., file I/O, network calls) in a controlled, composable way.
 - **ELIS**: Wrap effects in a function or container instead of performing them immediately. Basically, it is e.g. separating out parts of code that have side effects in a function, but still calling it all together when needed in a function.
 - This lets you test the functional parts of the code for testing and makes it easier to reason and more predictable.
 - Keeps the main program logic pure, deferr all effectful computations to a controlled context.
 - Ensures predictability, easier testing, and reasoning about program behavior.
- **Actor Model (Concurrent Systems)**
 - Everything is an actor, has independent private state, behavior, mailbox for message passing.
 - State mutations confined to actors, resembling ”imperative shell” around a functional core that processes messages.
 - Promotes safe concurrency (actors no shared mutable state), encapsulation, scalability.
- **Flux / Redux Architecture (Frontend State Management)**
 - Application state is immutable and updated by pure reducer functions (the functional core).
 - Have Store, Reducer, Action, Dispatcher, View UI. Reducers are the functional core, UI components, middleware, API calls form the imperative shell, all handled outside reducers.
 - Ensures predictable state transitions and simplifies testing, as the core logic is pure.
 - Originated in the React ecosystem, but is an architecture pattern for predictable state management.



Lecture 3.1: Objects and Data Structures

Data Abstraction

- **Goal:** hide representation and force clients to interact through meaningful operations.
- **Bad abstraction (data exposure):** A struct with public fields exposes implementation, lets clients mutate fields independently, and makes invariants hard to enforce.
- **Better abstraction (protocol interface):** Specify behavior in terms of operations, e.g. setCartesian and setPolar so coordinates are assigned consistently, and the representation (rectangular vs polar) can change.

```
struct Point {
    public var x = 0.0
    public var y = 0.0
}

protocol Point {
    var x: Double { get }
    var y: Double { get }
    mutating func setCartesian(x: Double, y: Double)
    var r: Double { get }
    var theta: Double { get }
    mutating func setPolar(r: Double, theta: Double)
}
```

- **Express in abstract terms:** prefer properties that state intent, not storage.
- E.g.: percentOfFuelRemaining hides tank size, internal units, can compute or cache.

```
protocol Vehicle {
    var fuelTankCapacityInGallons: Double { get }
    var gallonsOfFuel: Double { get }
}
```

```
protocol Vehicle {
    var percentOfFuelRemaining: Double { get }
}
```

Data/Object Anti-Symmetry

- **Objects:** hide data behind abstractions, expose functions that operate on that data.
- **Data structures:** expose data, have little to no behavior.
- These are complementary; mixing them blurs boundaries and weakens both.

Procedural vs Object-Oriented

- **Procedural (data + functions elsewhere):** easy to add new functions, but hard to add new data types (must update all functions).
- **Object-oriented (behavior with data):** easy to add new types, but hard to add new functions (must update all classes).
- “Everything is an object” is a myth; pick the style based on expected change.

```
// Procedural
struct Square { var topLeft: Point; var side: Double }
struct Rectangle { var topLeft: Point; var height: Double;
                  var width: Double }
struct Circle { var center: Point; var radius: Double }

struct Geometry {
    static let PI = 3.141592653589793
    func area(shape: Shape) throws {
        switch shape {
            case let s as Square: return pow(s.side, 2)
            case let r as Rectangle: return r.height * r.width
            case let c as Circle: return PI * pow(c.radius, 2)
            default: throw NoSuchShapeError
        }
    }
}
```

// Each time we add a new shape, we must update area()

```
// Object-Oriented
protocol Shape { var area: Double { get } }
```

```
struct Square: Shape {
    var topLeft: Point; var side: Double
    var area: Double { return pow(side, 2) }
}
struct Rectangle: Shape {
    var topLeft: Point; var height: Double; var width: Double
    var area: Double { return height * width }
}
struct Circle: Shape {
    static let PI = 3.141592653589793
    var center: Point; var radius: Double
    var area: Double { return PI * pow(radius, 2) }
}
// Each shape knows how to compute its own area
```

Law of Demeter (LoD)

- **Principle:** a module should not know the internals of the objects it manipulates.
- A method f of class C should only call methods of:
 - C itself
 - objects created by f
 - objects passed as arguments to f
 - objects held in instance variables of C

Train Wrecks

```
String outputDir = sctxt.getOptions().getScratchDir().getAbsolutePath();
```

- Long chains expose internal structure (many coupled train cars).
- Splitting can improve readability but not necessarily LoD compliance.

```
Options opts = ctxt.getOptions();
File scratchDir = opts.getScratchDir();
String outputDir = scratchDir.getAbsolutePath();
```

- Whether this violates LoD depends on what ctxt, Options, File are.
- If they are objects, asking for internals violates LoD. If they are pure data structures, LoD does not apply.

Hiding Structure vs Exploding APIs

- ctxt.getAbsolutePathOfScratchDirectoryOption() hides internals but can lead to method explosion.
- ctxt.getScratchDirectoryOption().getAbsolutePath() assumes returned value is a data structure.
- If ctxt is an object, we should **tell** it what we want done.

Single Level of Abstraction

- The code below mixes multiple levels: string paths, separators, file extensions, and I/O streams.
- This is hard to read and easy to break if details change.

```
String outFile = outputDir + "/" + className.replace('.', '/')
    + ".class";
FileOutputStream fout = new FileOutputStream(outFile);
BufferedOutputStream bos = new BufferedOutputStream(fout);
```

- Better: move the detailed steps behind a meaningful operation.

```
BufferedOutputStream bos =
    ctxt.createScratchFileStream(className);
```

Hybrids

- **Hybrid** classes mix object and data structure styles:
 - Have significant functions (object-like)
 - But also expose public fields or trivial getters/setters (data-like)
- Hard to add new functions and hard to add new data: worst of both worlds.
- Prefer to choose one side explicitly per type.

Conclusion

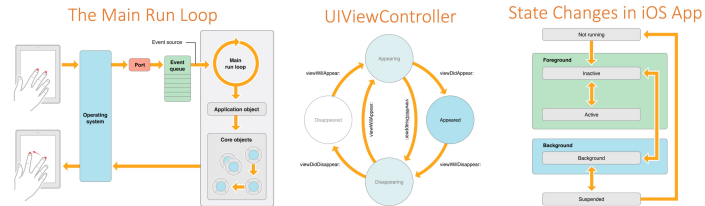
- **Objects** → object-oriented code.
- **Data structures** → procedural code.
- Good devs pick style based on likely changes, desired abstraction, without prejudice.

Lecture 4: Asynchronous Design Patterns

App Life Cycle and Run Loop

- The **main run loop** drives app execution: processes events and schedules work.
- In iOS, `@UIApplicationMain` wires up the application entry point via `AppDelegate`.
- The **main dispatch queue** is serial and runs on the main thread; only the main thread can update UI.

```
@UIApplicationMain
class AppDelegate: UIResponder, UIApplicationDelegate { ... }
```



UIViewController Lifecycle (Push ViewController A → VC B): Entry Process

- Lifecycle stages are predictable and should be used intentionally.
- **init / initWithCoder**: allocate resources and receive dependencies.
- **awakeFromNib**: nib loaded, outlets not fully laid out.
- **loadView**: create view hierarchy (override only if building UI in code).
- **prepareForSegue**: pass data before transition.
- **viewDidLoad**: called once; outlets connected; good for one-off setup or background work.
- **viewWillAppear**: called every time view appears; restart tasks that must be fresh.
- **viewWill/DidLayoutSubviews**: layout pass (constraints resolved).
- **viewDidAppear**: start animations, media, or data fetching.
- **didMoveToParentViewController**: completes parent-child transition.

UIViewController Lifecycle (Push VC B → VC C): B Exit Process

- **prepareForSegue**: trigger transition and pass data. (Segue: uninterrupted transition)
- **viewWillDisappear**: about to leave screen.
- **viewDidDisappear**: stop timers, notifications, sensors.
- **didReceiveMemoryWarning**: notifies low memory. release caches if possible.
- **deinit**: final cleanup before deallocation.

Why Use Asynchronous?

- **UI responsiveness**: long tasks on the main thread freeze UI.
- **Parallelism**: iOS devices have multiple cores; concurrent work can use them.
- **But**: concurrency is hard to reason about, costly to debug, and can slow code if abused.

Concurrency Pitfalls

- **Race conditions**: order of execution is not deterministic.
- **Reordering**: compiler or CPU may reorder instructions unless constrained.
- Result: counter-intuitive outcomes (e.g. two threads both reading old values).

```
// Thread 1           // Thread 2
x = 1                 r1 = y
y = 1                 r2 = x
// Possible: r1 = 0, r2 = 0 (init values as 0)
```

Threads vs Tasks

- **Thread**: a path of execution that shares memory with others.
- **Task**: abstract unit of work (often a closure in Swift).
- Avoid manual thread management (NSThread) unless absolutely necessary.

Critical Sections and Methods of Synchronisation

- **Critical section**: code that must not run concurrently.
- **Techniques**: atomic operations, memory barriers, volatile variables, locks.
- Swift provides limited low-level primitives; prefer higher-level frameworks.

```
// Non-atomic increment, vs Atomic (C / legacy APIs)
x += 1
OSAtomicIncrement32(&x)
```

Locks and Semaphores

- **Locks / mutexes**: mutual exclusion for critical sections.
- Variants: re-entrant, unfair, spin locks (tradeoffs in fairness vs latency).
- **Semaphores**: counting lock; used for producer-consumer.
- Pitfalls: accidental release, recursive deadlock, task-death deadlock, priority inversion.

Grand Central Dispatch (GCD) - fancy name for libdispatch

- GCD is the primary concurrency framework (libdispatch).
- In Swift, tasks to be carried out are closures, or function pointers, submitted to dispatch queues. GCD will call up a thread to service that task, tear down the thread when done.
- GCD handles thread pool management, scheduling, and context switching.
- **Dispatch queues** run tasks (closures).
- **Serial queue**: one task at a time. **Concurrent queue**: tasks start FIFO but run in parallel.
- **Two ways of dispatch**: **async**: submit work, return immediately. **sync**: submit work, block.

Types of Queues

- **Main queue**: serial, runs on main thread.
- **Global Background queues**: concurrent, QoS-based (high/default/low/background).
- **Custom queues**: serial or concurrent; ends up in one of global pools.

Quality of Service (QoS)

- use QoS to indicate task priority and resource needs, instead of priority, specified for tasks.
- **User-interactive**: immediate UI work, usually on main thread.
- **User-initiated**: user waiting, high priority.
- **Utility**: long-running work, energy efficient.
- **Background**: maintenance, prefetching. Tasks users not aware of.
- High QoS task will elevate queued tasks.

In Application Main Thread, has a main run loop that processes events and schedules work. The main dispatch queue is serial, runs on the main thread; only main thread can update UI.

Handling Events Asynchronously

- Do heavy work off the main thread, then return to main queue for UI updates.

```
let queue = DispatchQueue(label: "com.example.app")
queue.async {
    let smallImage = image.resize(to: rect)
    DispatchQueue.main.async {
        imageView.image = smallImage
    }
}
```

Structuring Concurrent Work in Applications

- Split by subsystems (UI, data transform, networking, database).
- Use queues per subsystem and keep data flow explicit.
- **Chaining**: tasks depend on previous tasks. **Grouping**: tasks run in parallel then join.

Dispatch Groups and Work Items

- **DispatchGroup**: track multiple tasks; use notify to join.
- **DispatchWorkItem**: attach flags, QoS, or cancellation behavior.

```
let group = DispatchGroup()
queue.async(group: group) { ... }
group.notify(queue: DispatchQueue.main) { ... }
```

Controlling Concurrency

- Too many concurrent tasks can cause **thread explosion**.
- Prefer async APIs and limit concurrency (serial queues, operation queues).
- **sync** on main or same queue risks deadlock.

Dispatch Barriers

- Dispatch Barriers allow you to make a concurrent queue act like a serial queue for a specific task. Wait for all previously submitted tasks to finish, then execute the barrier task exclusively, then allow subsequent tasks to proceed concurrently.
- Use on concurrent queues to isolate a write among reads. (Protect writes to shared data while allow concurrent reads). Gives reader-writer pattern without locks.
- Best used on a custom concurrent queue.

Swift Concurrency (Swift 5.5+)

- **async/await**: use async for functions, await marks suspension points explicitly.
- Asynchronous functions can only be called from other async contexts, `@main`, or child tasks.
- **Task**: unstructured child task with a handle for results. Allows you to call async API in synchronous code. (Wrap it in a Task, don't await, fire and forget).

```
func listPhotos(inGallery name: String) async -> [String] { ... }
let names = await listPhotos(inGallery: "Summer")
```

```
// unstructured child task, not tied to parent tasks lifetime
let handle = Task {
    return await add(newPhoto, toGalleryNamed: "Spring")
}
let result = await handle.value
```

- **Actors**: isolate mutable state; only one task can access actor mutable state at a time.

```
actor TemperatureLogger {
    let label: String
    var measurements: [Int]
    private(set) var max: Int
```

```
init(label: String, measurement: Int) {
    self.label = label
    self.measurements = [measurement]
    self.max = measurement
}
```

```
let logger = TemperatureLogger(label: "Outdoors", measurement: 25)
print(await logger.max) // access may suspend execution
```

- **Async sequences**: iterate values over time with `for try await`.

```
async let first = downloadPhoto(named: photoNames[0])
async let second = downloadPhoto(named: photoNames[1])
let photos = await [first, second]
```

Object Lifecycle in a Concurrent World

- Lifecycle pattern: **setup** → **activate** → **invalidate** → **deallocate**.
- Use explicit invalidation to unregister observers safely before deinit.
- Guard callbacks after invalidation to avoid races or use-after-free.
- **Why explicit invalidation?** Observers are often retained by the subsystem; unregistering in `deinit` can run on the wrong queue and cause deadlocks or leave references behind (leaks).
- **Rule of thumb**: invalidate on the correct queue (often main), then allow deallocation to happen later on a single thread.
- **In-flight callbacks**: even after unregistering, a callback may already be queued; check an invalidated flag in handlers.

```
class BusyController: SubsystemObserving {
    private var invalidated = false

    func invalidate() {
        dispatchPrecondition(.onQueue(DispatchQueue.main))
        invalidated = true
        DataTransform.sharedInstance.unregister(observer: self)
    }

    func systemStarted(...) {
        if invalidated { return }
        ...
    }

    deinit { precondition(invalidated) }
}
```

Summary

- Concurrency is powerful but risky; use frameworks and patterns.
- Dispatch queues make concurrency manageable, Swift 5.5+ makes it safer.
- Prefer clear lifecycle and explicit invalidation in concurrent observer designs.

Lecture 5: Entity Component Systems (ECS)

Why ECS for Game Architecture?

- Games have tight frame budgets and a **game loop** with ordered, repeated updates.
- OOP/POP often lead to deep hierarchies or tangled compositions when behaviors combine.
- ECS separates **data (components)** from **behavior (systems)** to scale complexity.

```
void update(int time) {
    game.update(time);
    spaceship.updateInputs(time);
    foreach (Enemy e : enemies) spaceship.move(time);
    foreach (Enemy e : enemies)
        foreach (Bullet b : bullets) collisionManager.update(time);
    spaceship.render(canvas);
    foreach (Enemy e : enemies) foreach (Obstacle o : obstacles) ...
    e.updateAI(time);
    e.move(time);
    b.move(time);
    e.render(canvas);
    o.render(canvas);
}
```

Process-Oriented Loop

- Use processes to break the loop into manageable stages (render, move, AI, collision).
- The command/process manager orders updates by priority.
- **Intuitive and good:** mirrors how humans think about game stages.
- **But:** still process-centric, can hard-code entity lists and duplicate data; ECS refines this by separating data (components) from behavior (systems).

```
interface IProcess {
    bool start();
    void update(int time);
    void stop();
}
```

```
class ProcessManager {
    private PriorityQueue<Process> processes = new PriorityQueue ...
    bool addProcess(Process process, int priority) { ... }
    void update(int time) { foreach(Process p : processes) p.update(time); }
    void removeProcess(Process process) { ... }
}
```

Example: Rendering as a Process

- Introduce IRenderable so the renderer depends on capability, not concrete types.
- Avoids hard-coding entity lists in rendering logic.
- **Why here?** Concrete example of splitting the big game loop into focused processes and shows dependency inversion: the render stage depends on an interface, not entities.

```
interface IRenderable { void render(Canvas canvas); }
```

```
class RenderProcess implements IProcess {
    private Canvas canvas;
    private Vector<IRenderable> renderables;
    void update(int time) {
        foreach(IRenderable r : renderables) r.render(canvas);
    }
}
```

Issue: Having to use an Inheritance Chain

- **DRY via base class** helps reuse (Renderable), but combining behaviors leads to multiple inheritance or awkward chains.
- Example problem: a Movable that is not Renderable.
- **BAD:** inheritance forces unrelated capabilities into one hierarchy, make types bloated, brittle.
- **BAD:** adding a new behavior can ripple through the whole tree (fragile base class).

```
class Renderable implements IRenderable { ... }
class Movable implements IMovable { ... }
class Spaceship extends Movable, Renderable { }
// not possible in most OO langs
```

Issue: What about Composition Over Inheritance?

- Compose behavior via delegates, but shared data becomes tricky (position, rotation).
- Interfaces alone provide methods, not shared state; you end up duplicating fields.
- **BAD:** duplicated state across delegates causes divergence bugs (render position $\neq$ move position).
- **OK:** composition avoids hierarchy explosion but still lacks a shared data model.

```
class Spaceship implements IRenderable, IMovable {
    Renderable renderDelegate;
    Movable moveDelegate;
    func render(canvas) { renderDelegate.render(canvas); }
    func move(time) { moveDelegate.move(time); }
}
```

Swift Semantics: Protocols Help, But Still Tight

- Swift protocols can require properties + default implementations.
- Still forces each entity to carry all fields for every capability.
- **BAD:** entity types grow as you add capabilities; unused fields are carried around.
- **BAD:** coupling between data and behavior remains at the entity level.

```
protocol Renderable {
    var sprite: Sprite { get set }
    var position: Point { get set }
    var rotation: Double { get set }
    func render(canvas: Canvas) -> ()
}
```

Architectural Pattern: Use Components

- Split state into small, reusable components.
- Entities are just IDs with a set of components.
- **GOOD:** data is normalized; no duplication between systems.
- **GOOD:** features are additive (add/remove components without changing types).

```
struct PositionComponent { var x: Double; var y: Double;
                           var rotation: Double }
struct VelocityComponent { var velocityX: Double; var velocityY: Double;
                           var angularVelocity: Double }
struct SpriteComponent { var sprite: Sprite }
```

Systems (formerly Processes)

- Systems operate on **sets of components**, not concrete entities.
- RenderSystem needs Position + Sprite; MoveSystem needs Position + Velocity.
- This avoids inheritance, favors data-oriented design, and is cache-friendly.
- **GOOD:** behavior scales with combinations of components, not class hierarchies.
- **GOOD:** easy to introduce new systems without touching entity definitions.

```
class RenderSystem: System {
    var renderables: [RenderData]
    func update(time: Int) {
        renderables.forEach { r in
            canvas.draw(r.sprite.sprite,
                        r.position.x, r.position.y,
                        r.position.rotation)
        }
    }
}
```

Steps to transition From Processes to Systems (Entity Component Systems)

- Start with data classes per process (RenderData, MoveData) $\rightarrow$ breaks encapsulation.
- Remove data classes; systems operate directly on components.
- Rename processes to **systems** once they are pure component processors.
- Entities become just **bags of components**.

Entity Explosion and Querying

- Entities can be created by assembling components at runtime.
- A Nexus (world) tracks entities and systems; systems query by component sets.
- Tradeoff: more indirection, but much more flexible composition.
- **Nexus as ECS extension:** centralizes storage, indexing, and scheduling so systems stay pure.
- **Why it helps:** avoids passing huge lists everywhere; queries become “give me all entities with Position+Velocity”.
- **Implementation intuition:** often a dictionary/map from entity IDs to component sets (or component type to entity data); high-performance ECS may use arrays/archetypes instead.

```
class Entity { var components = Set<Components>() }
```

```
class Nexus {
    var entities = Dictionary<UUID, [Components]>()
    var systems = PriorityQueue<System>()
    func getEntities(with components: [Components]) -> [[Components]] { ... }
    func update(time: Int) { systems.forEach { $0.update(time, self) } }
}
```

Summary

- ECS separates **entities** (IDs), **components** (data), and **systems** (logic).
- Great for games where behavior mixes combinatorially and performance matters.
- Costs: more plumbing and less encapsulation; use when flexibility outweighs that cost.

Lecture D: Protocol Oriented Programming

Why POP? What about OOP?

- OOP strengths: **encapsulation**, access control, abstraction, namespaces (logical grouping no name conflicts), expressive syntax, extensibility.
- Inheritance - reuse complex logic, allow overrides, but large hierarchies often become brittle.
- Most real systems want **composition** rather than deep inheritance trees.

Why Inheritance Hurts

- **Intrusive inheritance**: only one superclass; must inherit its stored properties and init rules.
- **Implicit sharing**: references lead to surprising mutations, defensive copying, locks, bugs.
- **Lost type relationships**: base types erase concrete info, forcing casts and runtime checks.

```
class Ordered {
    func precedes(other: Ordered) -> Bool { fatalError("Implement me!") }
}
class Number: Ordered {
    var value: Double = 0
    override func precedes(other: Ordered) -> Bool {
        return value < (other as! Number).value // forced downcast
    }
}
```

Protocol-Oriented Programming

- Start with a **protocol** before writing a class; model behavior instead of hierarchy.
- Use **value types** (structs) when possible to avoid implicit sharing.

```
protocol Ordered {
    // Self requirement: same concrete type as me
    func precedes(other: Self) -> Bool
}
```

```
struct Number: Ordered {
    var value: Double = 0
    func precedes(other: Number) -> Bool { value < other.value }
}
```

Two Worlds of Protocols

- **Without Self**: usable as a type (existential), heterogeneous collections, dynamic dispatch. Every model must deal with all others. Less optimisable.
- **With Self**: only usable as a generic constraint, homogeneous collections (all same type that implements protocol), static dispatch.

```
// Heretogeneous, cannot work with self (can't precedes() across types)
func binarySearch(sortedKeys: [Ordered], forKey k: Ordered) -> Int { ... }
```

```
// Generic, homogeneous
func binarySearch<T: Ordered>(sortedKeys: [T], forKey k: T) -> Int { ... }
```

Renderer Example (Protocols + Generics)

- Define Drawable and a Renderer protocol to separate drawing intent from rendering back-end.
- Enables testing via a `TestRenderer` and real rendering via `CoreGraphics`.
- **Retroactive modeling**: extend existing types to conform without modifying them.

```
protocol Renderer {
    func moveTo(position: CGPoint)
    func lineTo(position: CGPoint)
    func arcAt(center: CGPoint, radius: CGFloat,
               startAngle: CGFloat, endAngle: CGFloat)
}
```

```
protocol Drawable { func draw(renderer: Renderer) }
```

```
struct Circle: Drawable { ... }
struct Polygon: Drawable { ... }
```

```
// extend Swift's CGContext to conform to Renderer
extension CGContext: Renderer { ... } // retroactive modeling
```

Protocol Extensions

- Provide **default implementations** and new APIs without changing conforming types.
- Extensions can be **constrained** (e.g., only when `Self: Equatable`).
- Customization points: default behavior can be overridden by conforming types.

```
extension Renderer {
    func circleAt(center: CGPoint, radius: CGFloat) {
        arcAt(center: center, radius: radius, startAngle: 0, endAngle: twoPi)
    }
}
```

```
extension Ordered where Self: Comparable {
    func precedes(other: Self) -> Bool { self < other }
}
```

Bridging Heterogeneous Types

- Protocols with `Self` cannot be used as heterogeneous element types.
- Bridge with a type-erased method (e.g., `isEqualTo`) to compare different Drawables.

```
protocol Drawable {
    func draw(renderer: Renderer)
    func isEqualTo(other: Drawable) -> Bool
}
extension Drawable where Self: Equatable {
    func isEqualTo(other: Drawable) -> Bool {
        if let o = other as? Self { return self == o }
        return false
    }
}
```

Generics vs Associated Types

- **Generics**: define a concrete implementation over a type parameter.
- **Generics expand implementation**: a generic function/type is one implementation that works for many types.
- **Associated types**: protocols specify placeholders to be filled by conforming types.
- **Associated types expand vocabulary**: a protocol is a checklist of requirements plus a named type slot. (A protocol with a 'type' slot.)

```
// Associated type
protocol Container {
    associatedtype T
    mutating func append(_ item: T)
    var count: Int { get }
    subscript(i: Int) -> T { get }
}
```

```
// Associated types filled by conformers
class BagOfBytes: Container {
    func append(_ item: UInt8) { ... }
    subscript(i: Int) -> UInt8 { ... }
}
```

```
struct BunchOfEquatables<T: Equatable>: Container {
    func append(_ item: T) { ... }
    subscript(i: Int) -> T { ... }
}
```

```
// Generic struct
struct Stack<T>: Container { ... }
```

```
// Generic type parameters (one implementation, many types)
class ViewController<V: View> { var view: V }
```

```
func allEqual<T: Equatable>(a: T, b: T, c: T) -> Bool {
    return a == b && b == c
}
```

When to Use Classes

- Use classes when implicit sharing is desired or required by frameworks.
- Examples: UI objects, file handles, sinks to external state (e.g., `CGContext`).
- Don't fight the system, if a framework expects a class, use a class. On the other hand, be circumspect, nothing should grow too large, when factoring something out of a class, consider making it a struct + protocol instead.
- Otherwise, prefer structs + protocols for composition and testability.

Swift Protocols vs Java Interfaces

- Swift protocols can require initializers, static methods, operators, and subscripts.
- Conditional conformances and constrained extensions enable expressive reuse.
- Default implementations and retroactive modeling support composition over inheritance.

Composition Over Inheritance

- Protocols as composition: Favor small, focused protocols over large monolithic ones.
- Avoid “god classes” by splitting features into protocols and delegates.
- Use adapters/decorators/proxies to compose behavior without inheritance chains.

Summary

- Protocols + value types + extensions are the core of Swift POP.
- Prefer composition and generic constraints to reduce hierarchy complexity.
- Use classes intentionally when identity and shared mutable state are essential.

Lecture E: Errors and Exceptions

Context and History

- Cocoa / Objective-C legacy influences Swift error handling (type bridging, NSError APIs).
- Objective-C initially lacked native exceptions; `NSException` was added via macros and reserved for catastrophic, unrecoverable errors.
- `NSError` predates exceptions and is used for recoverable, user-facing failures.

Errors vs Exceptions

- **NSError**: recoverable, expected failures (e.g., file not found).
- **NSException**: programmer/runtime errors (e.g., out-of-bounds), not meant for control flow.
- Swift introduces **Error** and **throw/try/catch** with explicit handling.

Swift Error Handling

- **Define**: create error types conforming to `Error`.
- **Throw**: mark functions with `throws`.
- **Handle**: use `do-try-catch`, `try?`, or `try!`.
- **Propagate**: a function can `throws` if it only fails when a callee fails.

```
enum FileError: Error { case notFound, unreadable }
```

```
func load(path: String) throws -> Data { ... }
```

```
do {  
    let data = try load(path: "notes.txt")  
} catch {  
    // handle  
}
```

Handling Options

- **do-try-catch**: explicit recovery paths.
- **try?**: converts errors to `nil` (optional result).
- **try!**: disables error handling; crashes if an error is thrown.
- **defer**: runs cleanup code on exit, even with errors.

Checked vs Unchecked

- Java has checked and unchecked exceptions.
- Swift has no unchecked exceptions; all **throw** sites must be acknowledged.
- Runtime traps (e.g., out-of-bounds) still exist and terminate execution.

Why Not Special Return Values?

- Special values are ambiguous and may collide with valid results.
- They carry limited error information unless wrapped in a custom type.
- Swift errors are effectively **typed return values** with syntactic sugar.

Performance Notes

- Exceptions are expensive in many languages due to stack unwinding and binary bloat.
- Swift favors explicit error paths over hidden runtime costs.

Interacting with Cocoa

- Swift bridges to Objective-C APIs that use `NSErrorPointer`.
- Swift 2.0 introduced automatic conversion of many Cocoa APIs to **throws**.
- Legacy APIs still expose `NSError` and must be handled explicitly.

Summary

- Swift error handling is explicit, typed, and recoverable.
- Exceptions are for programmer errors; use **Error** for expected failures.
- Cocoa legacy shapes the API surface; bridging is common in real codebases.

Lecture F: Requirements and Specifications

Motivation: Requirements Drive Design

- A game engine focuses on gameplay logic and state; a physics engine focuses on motion and collision.
- Requirements shape decomposition: model/view/controller vs physics subsystems.
- Composition (protocols/decorators) can avoid rigid inheritance as features like gravity, forces, or special effects grow.

Why Requirements Matter

- Wrong requirements and late integration are expensive to fix.
- Requirements change; plan for evolution rather than locking into a brittle design.
- Users often cannot articulate needs directly; you must uncover the real problem.

Software Development Life Cycle (SDLC)

- **Waterfall**: sequential phases; clear milestones but rigid to change.
- **Spiral**: iterate with risk analysis; suitable for high-uncertainty projects.
- **Agile**: iterative, incremental delivery with frequent feedback loops.

Requirements Analysis

- Understand workflow and user interactions; remember users do not think like developers.
- Consider failure modes: what happens when things go wrong?
- Capture **functional** requirements, **performance** constraints, and **delivery** schedule.

Specifications and Diagrams

- Specifications define operations and constraints for static or running systems.
- Diagrams clarify structure: ERD for entities/relations, MDD for class/module interactions.
- A picture can surface missing constraints earlier than prose alone.

Design Goals and Decomposition

- Aim for modular structure with clear abstractions and independent implementations.
- Decompose to reduce complexity, divide work, improve flexibility and reuse.
- Prefer high cohesion and low coupling; use design patterns to guide interfaces.

Interfaces vs Implementation

- A module's API is the contract; implementation can change without breaking clients.
- Phased implementation helps manage scope: build in small, testable increments.
- Design for change: think big, implement in small steps.

Final Project Guidance

- Focus on software engineering, not just flashy features.
- Plan: data model, design (ERD/MDD), specifications, and usability.
- Implement with ADTs, clear dependencies, unit tests, and staged integration.
- Requirements may change midstream; reconcile external needs with course goals.